

A HOPEFUL TALK FOR FIRST- AND SECOND-YEAR CSE STUDENTS

The Beginner's Advantage in the Age of AI

AI is making coding more accessible. That is exactly why fundamentals, critical thinking, and curiosity are becoming more valuable.

Vibhor

VP Engineering, CareSmartz360

Chitkara University '08

Same student energy, very different tools



Graduation, 2008: a beginner with questions.

2008



2026



Today: same curiosity, much stronger leverage.

The point of this talk is possibility: the same curiosity now has dramatically more leverage.

A better way to read the AI moment

AI makes first drafts easier. That gives beginners more room to practice thinking, building, and asking sharper questions.

Surface reading

- AI can generate code quickly.
- Many tools now start from natural language.
- Beginners can prototype earlier.

Deeper reading

- Thinking shapes what is worth building.
- Fresh minds can form strong habits early.
- Students can learn by making real things.

Build sooner

Use AI to make ideas testable earlier.

Think deeper

Ask what would make the answer reliable.

Learn visibly

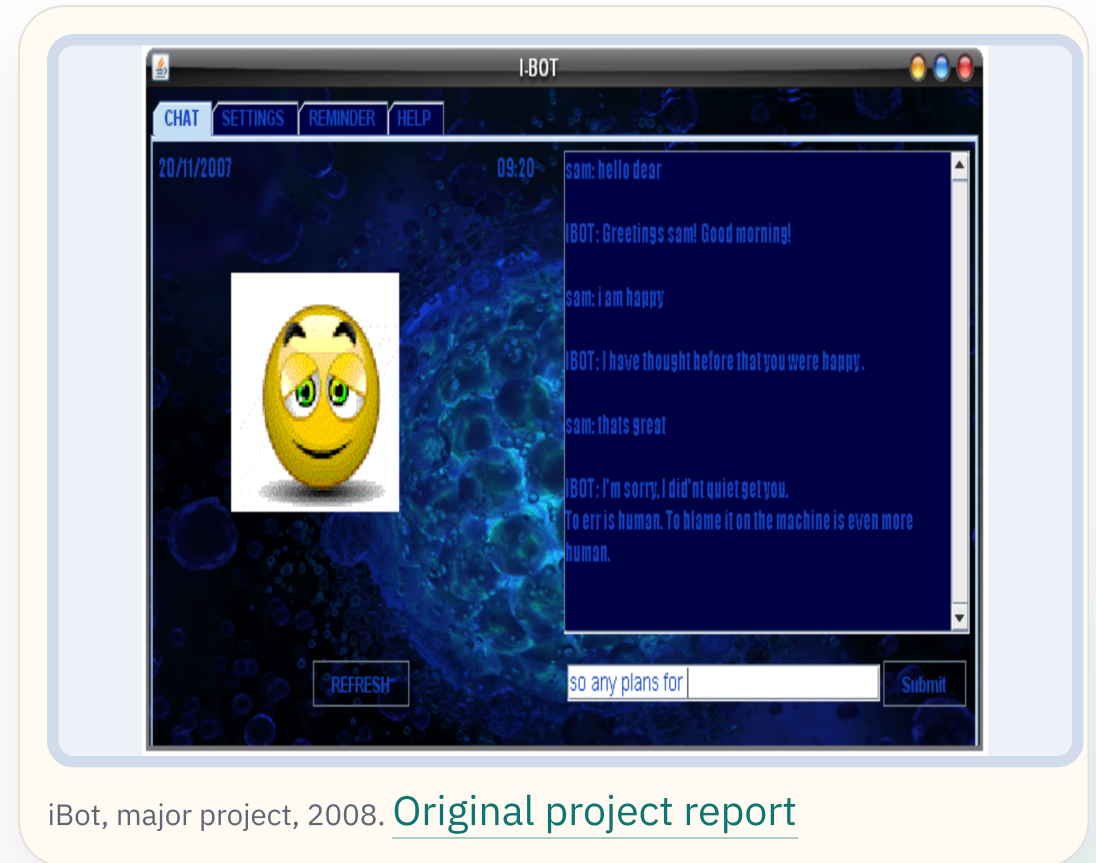
Turn theory into small working systems.

In 2008, my “AI” project looked like this

I built an AI chatbot as my final-year project at the same college, long before today’s LLM wave.

- The ambition was real, and the constraints taught patience.
- Trying ideas like this in 2008 required far more time and setup.
- What changed most by 2026 is the cost of experimentation.

This artifact is a reminder: the urge to build was already there. Today, students have much more leverage to act on it.



iBot, major project, 2008. [Original project report](#)

What changed since 2008, and what endured

Changed

- Boilerplate is cheaper.
- Prototypes are faster.
- Trying ideas is dramatically easier.
- AI can assist across the workflow.

Endured

- Correctness still depends on mental models.
- Users still depend on our choices.
- Systems still hide assumptions.
- Curiosity and judgment still compound.

Syntax

Easier to generate.

Framing

Harder to fake.

Judgment

More valuable now.

Curiosity

Still the edge.

CareSmartz Lite showed what beginners can do

Freshers contributed to real product work when they had context, trust, and useful tools.

- They learned fast when the work was framed well.
- They asked better questions as they gained context.
- They improved quickly when trust and tooling were present.

The beginner's advantage is learning velocity: curiosity, openness, and fast feedback loops.

Context

Explain why the problem matters.

Trust

Give real ownership early.

Tooling

Good guardrails turn effort into progress.

Feedback

Fast loops create confidence.

When the path changed, the team learned and delivered

Learn fast

I saw this when the team shipped AI features by learning quickly and taking ownership of the problem.

Test assumptions

- The team learned Python quickly.
- They built capability through experiments and review.
- They improved model choices through evidence.

Correct course

Readiness grows through movement: learn, try, review, improve, and deliver.

Deliver

Engineering work is now humans plus AI agents

RCA

Connect symptoms to causes.

Patch + TDD

Generate, verify, and review with evidence.

Architecture Review

Compare options and expose trade-offs.

Code Review

Catch patterns, risks, and drift.

Incident Review

Turn incidents into learning loops.

Customer Briefing

Translate complexity into clear language.

AI now extends across debugging, testing, review, documentation, and communication.

Still human-owned

- Correctness
- Privacy
- Bias and safety
- Consequences

The better the human judgment, the better the AI outcome.

THE DEEPER OPPORTUNITY AI CREATES

Faster tools make fundamentals more valuable

Speed makes deep understanding
more valuable.

Classroom topics become product superpowers

Data Structures

Scheduling conflicts reveal interval logic inside calendar UI.

Databases

Indexes, compatibility, and assumptions decide real outcomes.

Operating Systems

Processes, memory, boot, and isolation become concrete ideas.

Networks

Reliability, latency, ordering, and retries are lived trade-offs.

Testing

Testing is structured doubt for both humans and AI.

Architecture

Alternatives, boundaries, and operational cost remain central.

As AI makes code easier to generate, CSE becomes a training ground for thinkers, builders, and responsible engineers.

Real examples: fundamentals turn into judgment

Scheduling conflicts reveal DSA

- Intervals, unions, intersections, recurrence.
- A caregiver calendar is also an algorithm problem.
- Abstraction helps clarify rich product reality.

SQL behavior reveals assumptions

- A compatibility issue made environmental expectations visible.
- Systems that look similar can behave differently.
- AI helps more when the engineer understands the ground truth.

In real engineering, visible behavior is an invitation to understand the deeper system.

2026 is the best time to be a maker

For the first time, students can actually live the lessons while they are still learning them.

2008 reality

- Ambitious ideas required more setup time.
- Curiosity often had to wait for tooling.
- Many concepts were harder to experience directly.

2026 reality

- AI lowers the cost of experimentation.
- You can build while learning theory.
- Making has become a practical way to study.

This is one of the best times in history to become a maker because curiosity can turn into working systems faster than ever.

If you are learning operating systems, build a tiny world

Live the lesson

- Vibe-code a tiny hello-world flavor of Linux from scratch.
- Let boot, memory, processes, and interrupts become felt experiences.
- Use AI to accelerate the scaffolding while you stay close to the meaning.

Boot

How does anything start?

Memory

Where does state live?

Process

What is actually running?

I/O

How does the machine talk back?

In 2008, this was intellectually exciting and time-intensive. In 2026, it is practical enough to try while you learn.

If you are learning networks, send real packets

Make the trade-offs visible

- Build a tiny app that sends data over TCP and UDP.
- See reliability, ordering, retries, and speed as behavior you can observe.
- Watch the concept become real as the system responds in front of you.

TCP

Reliable, ordered, heavier.

UDP

Lighter, faster, no guarantees.



The concepts become memorable when you can see packets, delays, and trade-offs with your own eyes.

How to build your beginner's advantage now

Practice like this

1. Start from first principles before you reach for prompts.
2. Build small, personal, useful systems while you study.
3. Use AI as a thought partner while you keep reasoning active.
4. Write tests, explain trade-offs, and keep evidence.
5. Ask better questions than your peers.

Own your judgment

When you can explain it, you can improve it.

Respect boundaries

Know what data you are sharing and why.

Prefer useful constraints

Good conventions and strong frameworks help both humans and AI stay honest.

tiny OS hello world

TCP vs UDP mini app

schedule conflict visualizer

tamper-evident notes

TAKEAWAY

Fresh minds are arriving at exactly the right time.

The field still needs beginners who can think clearly, build courageously, and turn faster tools into trustworthy systems.

Thinkers

Builders

Responsible engineers

Chitkara 2008 to 2026: this may be the best time yet to learn by making.